CO1 – ASSIGNMENT 1

Design, Implement and Validate a Reliable Billing Program for a Rural Electricity Co-operative

Student: Sudharsan R **Register No.:** 1912260284

Course: C Programming **Submission date:** 27-10-2026

TASK 1 – Understand the Problem and Identify the Data (4 Marks)

1. Inputs, outputs and stored data

Inputs: household/meter number (identifier), previous meter reading, current meter reading, and consumer category if different categories use different tariffs. For a basic implementation, the category is not used because the assignment supplies one tariff. The readings are integers from a 16-bit meter register, 0–65535.

Outputs: units consumed, fixed charge, energy charge for each slab, total bill in paise, and formatted total in rupees. Keeping the bill in paise prevents fractional-paise rounding errors.

Constants: register modulus (65536), fixed charge (5000 paise), free slab limit (100 units), second slab limit (200 units), and the slab rates (0, 225, and 450 paise per unit). These values remain unchanged during one execution and should be defined centrally.

2. Validity rules and assumptions

• Meter readings must be between 0 and 65535 inclusive.

• If current reading is greater than or equal to previous reading, consumption = current – previous.

• If current reading is less than previous reading, assume the 16-bit register rolled over once: consumption = (65536 – previous) + current.

• Consumption, each charge and the final total must never be negative; total bill must equal fixed charge plus all slab charges.

• Assume at most one rollover occurs between readings. If a meter can roll over more than once, timestamps or rollover counters are needed.

• The fixed charge applies even when consumption is zero, as shown by the supplied expected output.

3. Alternative representation and complexity

If readings are stored as text (for example, "01530"), the program must validate that every character is a digit and convert the string to an integer. For a d-digit reading, conversion and validation take O(d) time. The string needs O(d) bytes plus its terminating null character, while a 16-bit integer needs a fixed 2 bytes on the target system. Integer arithmetic is simpler and avoids repeated conversion. The billing calculation itself is O(1) time and O(1) extra space because it performs a fixed number of arithmetic operations.

Evidence expected: the input/output list, assumptions and validity rules above identify what must be stored and checked.

TASK 2 – Choose the Data Types (4 Marks)

1. Recommended data types

Data item	Suggested type	Reason
Meter readings	uint16_t (or unsigned short after checking limits.h)	Exactly 16 bits when uint16_t exists; range 0–65535.
Consumption	unsigned int / uint32_t	Can represent 0–65535 units; wider type also allows future expansion.
Tariff constants and charge	long / int64_t for paise	Integer paise avoids floating-point rounding; use a wide type for totals.

Data item	Suggested type	Reason
Bill output	long long or int64_t	Safe for a large number of units and accumulated bills.
Household/meter number	string or suitable unsigned integer	An identifier is not used for arithmetic; text preserves leading zeros.
Category	enum	Makes named categories clearer and reduces accidental invalid values.

The actual size of C types is implementation-dependent. Use sizeof and limits.h to verify size and range on the compiler/system used. A 16-bit unsigned register has maximum 65535; adding 1 in a 16-bit register wraps to 0. A signed 16-bit short cannot represent all meter readings.

2. Why not float rupees?

Binary floating-point cannot represent many decimal fractions exactly. If thousands of bills are added, small representation errors can accumulate and rounding may occur at different stages. Store money as integer paise, calculate exactly, and convert only when displaying: rupees = paise / 100 and paise remainder = paise % 100.

3. Complete code: 16-bit meter rollover

```
#include <stdio.h>
#include <limits.h>

typedef unsigned short meter_t;

unsigned int consumption(meter_t previous, meter_t current)
{
    /* The assignment specifies a 16-bit meter register. */
    if (current >= previous)
        return (unsigned int)current - (unsigned int)previous;

    /* One rollover: (65535 -> 0), modulus is 65536. */
    return (65536U - (unsigned int)previous) + (unsigned int)current;
}

int main(void)
{
    printf("sizeof(meter_t) = %zu bytes, maximum reading = %u\n",
           sizeof(meter_t), (unsigned int)USHRT_MAX);
    printf("Consumption: %u units\n", consumption(1200, 1530));
    printf("Consumption: %u units\n", consumption(65400, 120));
    return 0;
}
```

Expected output on a system with 16-bit unsigned short:

```
sizeof(meter_t) = 2 bytes, maximum reading = 65535
Consumption: 330 units
Consumption: 256 units
```

The second result is 256 because the register counts 65400 to 65535 (136 counts), wraps to 0, then counts to 120: 136 + 120 = 256. The calculation uses unsigned wider arithmetic, so it does not produce a negative value. If unsigned short is not 16 bits on the compiler, use uint16_t from and verify that it is available.

Alternative concept and complexity

If the register value were stored as a double rupee amount rather than integer paise, each addition could introduce a small binary floating-point error. Integer paise gives exact addition for whole paise values. Both approaches take O(1) time per bill and O(1) space per bill, but integer paise is exact within the chosen integer range.

TASK 3 – Define the Constants (3 Marks)

1. Constant definitions and complete program

```
#include <stdio.h>

#define SLAB1_UNITS 100
#define SLAB2_UNITS 200
#define RATE1_PAISE 0       /* first 100 units are free */
#define RATE2_PAISE 225     /* Rs 2.25 per unit */
#define RATE3_PAISE 450     /* Rs 4.50 per unit above 200 */
#define FIXED_CHARGE_PAISE 5000L  /* Rs 50.00 */

long long compute_bill(int units)
{
    long long paise = FIXED_CHARGE_PAISE;

    if (units <= SLAB1_UNITS) {
        paise += (long long)units * RATE1_PAISE;
    } else if (units <= SLAB2_UNITS) {
        paise += (long long)(units - SLAB1_UNITS) * RATE2_PAISE;
    } else {
        paise += (long long)(SLAB2_UNITS - SLAB1_UNITS) * RATE2_PAISE;
        paise += (long long)(units - SLAB2_UNITS) * RATE3_PAISE;
    }
    return paise;
}

int main(void)
{
    int tests[] = {0, 100, 150, 200, 350};
    int n = (int)(sizeof tests / sizeof tests[0]);

    for (int i = 0; i < n; i++) {
        long long p = compute_bill(tests[i]);
        printf("%3d units -> Rs %lld.%02lld\n",
               tests[i], p / 100, p % 100);
    }
    return 0;
}
```

Expected output:

```
  0 units -> Rs 50.00
100 units -> Rs 50.00
150 units -> Rs 162.50
200 units -> Rs 275.00
350 units -> Rs 950.00
```

2. Why use #define, const and enum?

#define: preprocessor text replacement; it has no variable storage and no C type checking. Use uppercase names for macros. **const:** a typed read-only object that can be inspected in a debugger and may occupy storage. **enum:** best for a set of named integral choices such as consumer categories. For these fixed tariff numbers, macros meet the task requirement; a const-based design can improve type checking and debugging. Changing RATE2_PAISE from 225 to 230 in one definition updates every calculation that uses the second slab. Runtime cost is effectively unchanged.

Alternative concept and complexity

Using const variables instead of #define gives typed values and easier debugging, but may allocate storage depending on the compiler and use. An enum is appropriate for categorical values, not ideal for currency rates requiring a specific type. The slab calculation performs a fixed number of comparisons and arithmetic operations: O(1) time and O(1) extra space.

TASK 4 – Use Macros Safely (4 Marks)

1. Safe macro definitions

```
#include <stdio.h>

#define RUPEES_TO_PAISE(r) ((long long)(r) * 100LL)
#define SQUARE(x)          ((x) * (x))
#define MAX(a, b)          (((a) > (b)) ? (a) : (b))

/* Prefer a function for expressions with side effects. */
static inline int max_int(int a, int b)
```

```
{
    return (a > b) ? a : b;
}

int main(void)
{
    int x = 5;
    int y = 9;

    printf("Rs 12 in paise = %lld\n", RUPEES_TO_PAISE(12));
    printf("Square of %d = %d\n", x, SQUARE(x));
    printf("Maximum = %d\n", MAX(x, y));
    printf("Function maximum = %d\n", max_int(x, y));
    return 0;
}
```

Expected output:

```
Rs 12 in paise = 1200
Square of 5 = 25
Maximum = 9
Function maximum = 9
```

2. Macro pitfalls and how to avoid them

• **Parentheses:** define SQUARE(x) as ((x) * (x)), not x*x. Without parentheses, SQUARE(2+3) may expand to 2+3*2+3 and give 11 instead of 25.

• **Repeated evaluation:** MAX(a,b) may evaluate one argument more than once. MAX(i++, j++) can increment a variable multiple times and is unsafe. Use a function such as max_int when arguments may have side effects.

• **Preprocessor expansion:** gcc -E source.c shows expanded source; gcc -E -P source.c removes line markers. gcc -E is evidence that macros are expanded before compilation.

• **Function versus macro:** an inline function provides type checking, evaluates each argument once, and is easier to debug. A macro may avoid a function-call cost, but modern compilers often inline small functions.

3. Complexity comparison

For a macro used k times, the expanded expression is copied into the program at each use, so generated source/object code may grow with k. A function has one function definition and a call at each use; if it is inlined, the compiler may also duplicate the code. For simple arithmetic, both have O(1) time per use and O(1) working space; the key differences are code size, type safety, debugging and repeated argument evaluation. A preprocessor macro itself has no run-time call overhead, but it is not automatically faster than an optimised inline function.

4. Compilation and validation commands

```
gcc -std=c11 -Wall -Wextra -pedantic task2.c -o task2
./task2
gcc -std=c11 -Wall -Wextra -pedantic task3.c -o task3
./task3
gcc -std=c11 -Wall -Wextra -pedantic task4.c -o task4
./task4
gcc -E -P task4.c > task4_preprocessed.c
```

The code snippets are separate programs; save each in its own file (task2.c, task3.c and task4.c) before compiling. For the rollover code, the displayed sizeof output assumes unsigned short is 2 bytes and has maximum 65535 on the target system.

Conclusion

A reliable billing program should validate readings, explicitly handle one meter rollover, store money as integer paise, centralise tariff constants, and use fully parenthesised macros only where appropriate. The supplied test cases verify ordinary consumption, rollover, tariff slabs and macro calculations.